

Flush — Onchain Poker Protocol Architecture

Legend

Symbol / Style	Meaning
 Rectangle	On-chain account / PDA (program-owned state)
 Rectangle	Off-chain service (server, frontend)
 Rounded	Instruction / process step
 Diamond	Decision point (branching / validation)
 Hexagon	Oracle (Switchboard VRF)
 Cylinder	Database (Postgres / Valkey)
 Cloud	External managed service (Privy, Helius, Vercel)
 solid arrow	Control flow / instruction call
 dashed arrow	Data flow / event / read
CPI	Cross-Program Invocation

0. Protocol Requirements

The protocol shall:

1. Allow an operator to **initialize a global config** (admin authority, rake basis points, treasury vault).
2. Allow a round to be **started with a unique round_seed** (each round is a distinct PDA).
3. Allow authenticated players (Privy embedded wallets) to **deposit SOL buy-in** into a per-round **pot vault**, creating a **PlayerSeat** .
4. Use a **Switchboard VRF** to generate verifiable randomness that deals **5 community/table cards** and **2 hole cards per player** from a 52-card deck.
5. Drive betting (check / call / raise / fold) across the four streets (pre-flop, flop, turn, river) — orchestrated by the game server, settled on-chain.
6. On round end, **evaluate the best 5-card hand**, split the pot between winner(s), and **charge a rake** to the treasury.
7. **Close** round/seat accounts and return rent once settled.
8. Run **all transactions in the background** — no user signing prompts — via Privy server-delegated signing.

1. System Context — End-to-End Architecture

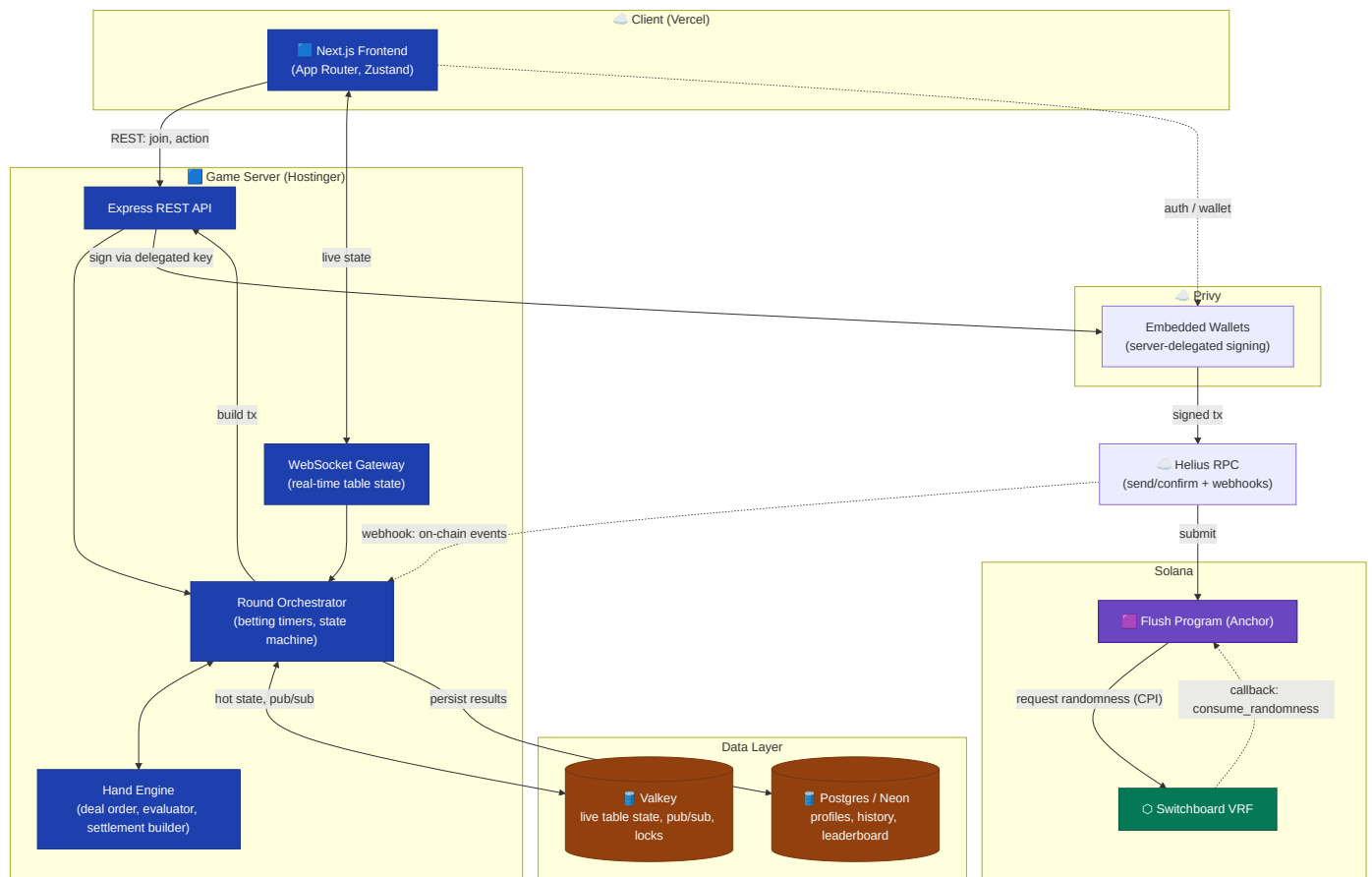


Figure 1 — System Context: end-to-end architecture

Separation of concerns

- **On-chain (trust):** custody of funds (pot vault), verifiable randomness, authoritative settlement & rake.
- **Server (orchestration):** betting timers, turn order, hand evaluation, transaction construction, real-time fan-out. Holds **no custody** — it can only move funds through program instructions the players' delegated keys sign.
- **Data layer:** Postgres = durable record of truth for history/profiles; Valkey = ephemeral hot-path state and cross-instance pub/sub.

2. Program Structure & Account Model

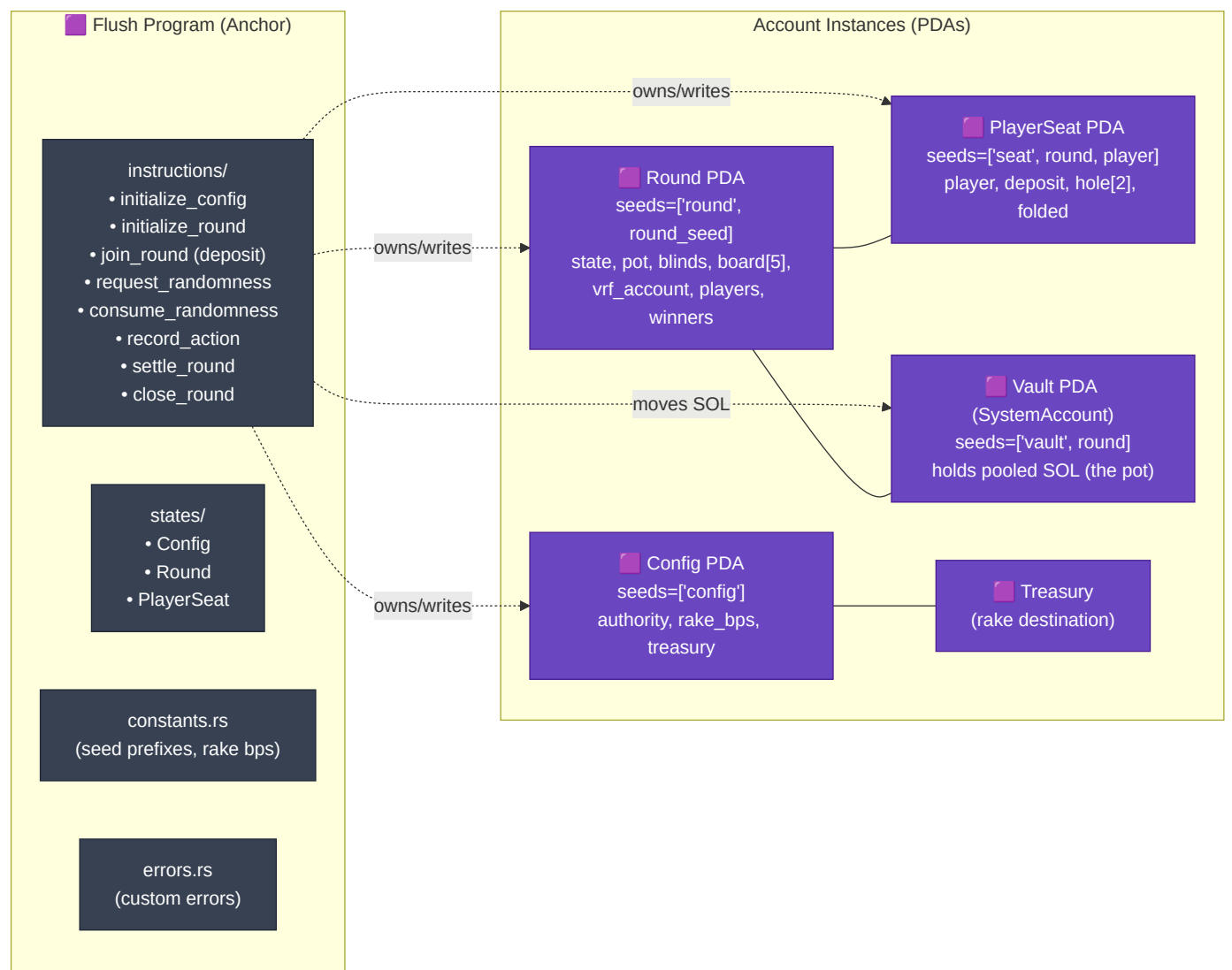


Figure 2 — Program structure & account model

Account responsibilities

Account	Type / Owner	Seeds	Key data
Config	PDA / Flush program	["config"]	authority, rake_bps, treasury, bump
Round	PDA / Flush program	["round", round_seed]	round_seed, state, pot, small_blind, big_blind, seat_count, board[5], vrf_account, winners, created_at, bump
PlayerSeat	PDA / Flush program	["seat", round_key, player]	player, deposit, hole[2], folded, all_in, bump
Vault	PDA (SystemAccount)	["vault", round_key]	Pooled SOL = the pot; only program signs withdrawals
VRF	Switchboard-owned	external	Randomness buffer; referenced by Round.vrf_account

Round state machine

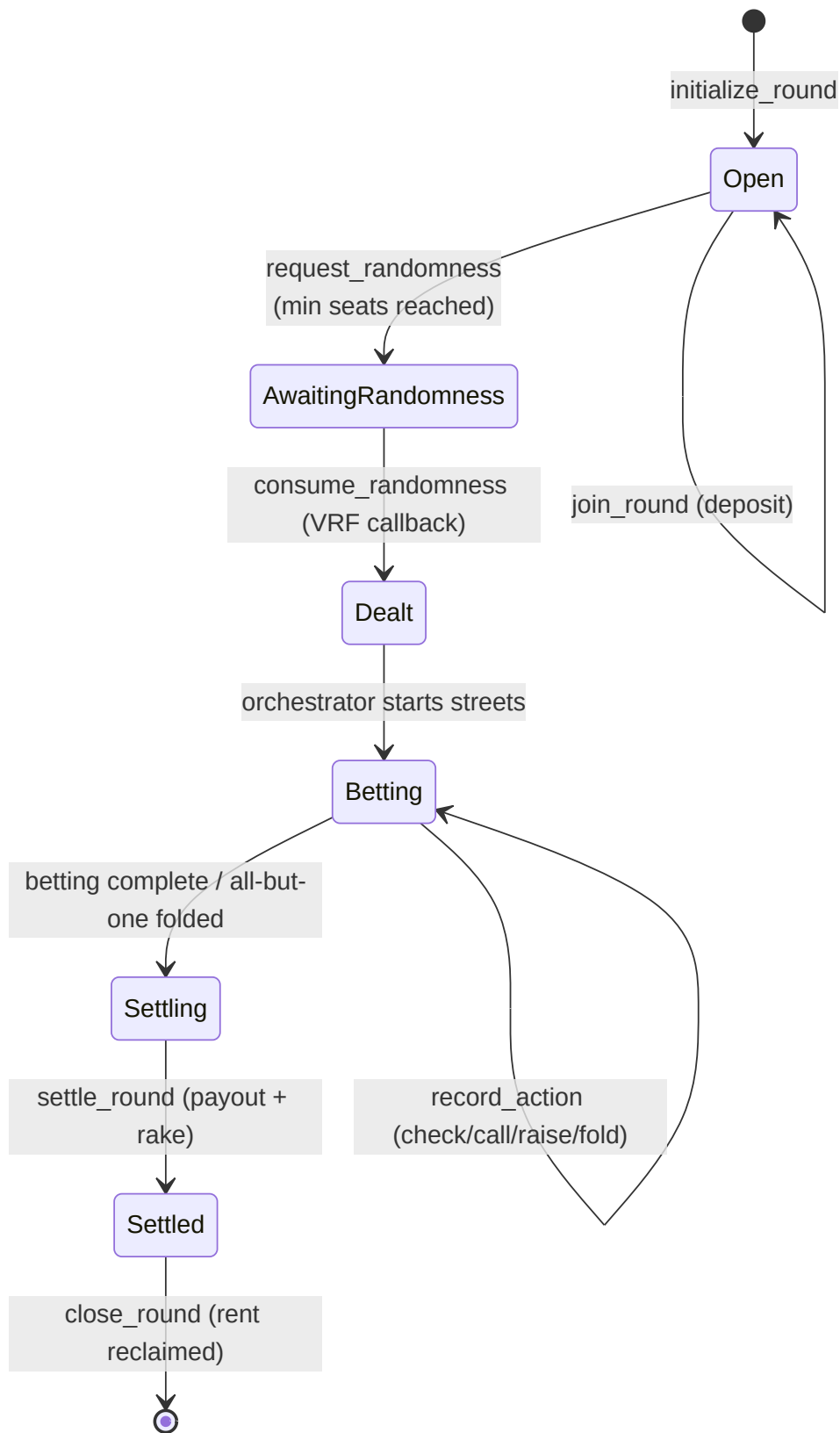


Figure 3 — Round state machine

3. PDA Derivation Map

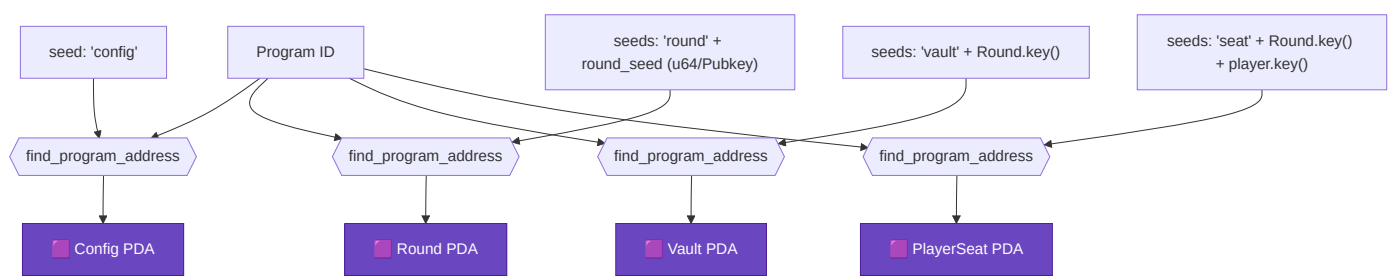


Figure 4 — PDA derivation map

`round_seed` is generated by the orchestrator per game (e.g. a monotonic counter or random u64 persisted in Postgres) so every round is a fresh, collision-free PDA.

4. End-to-End Round Lifecycle (Sequence)

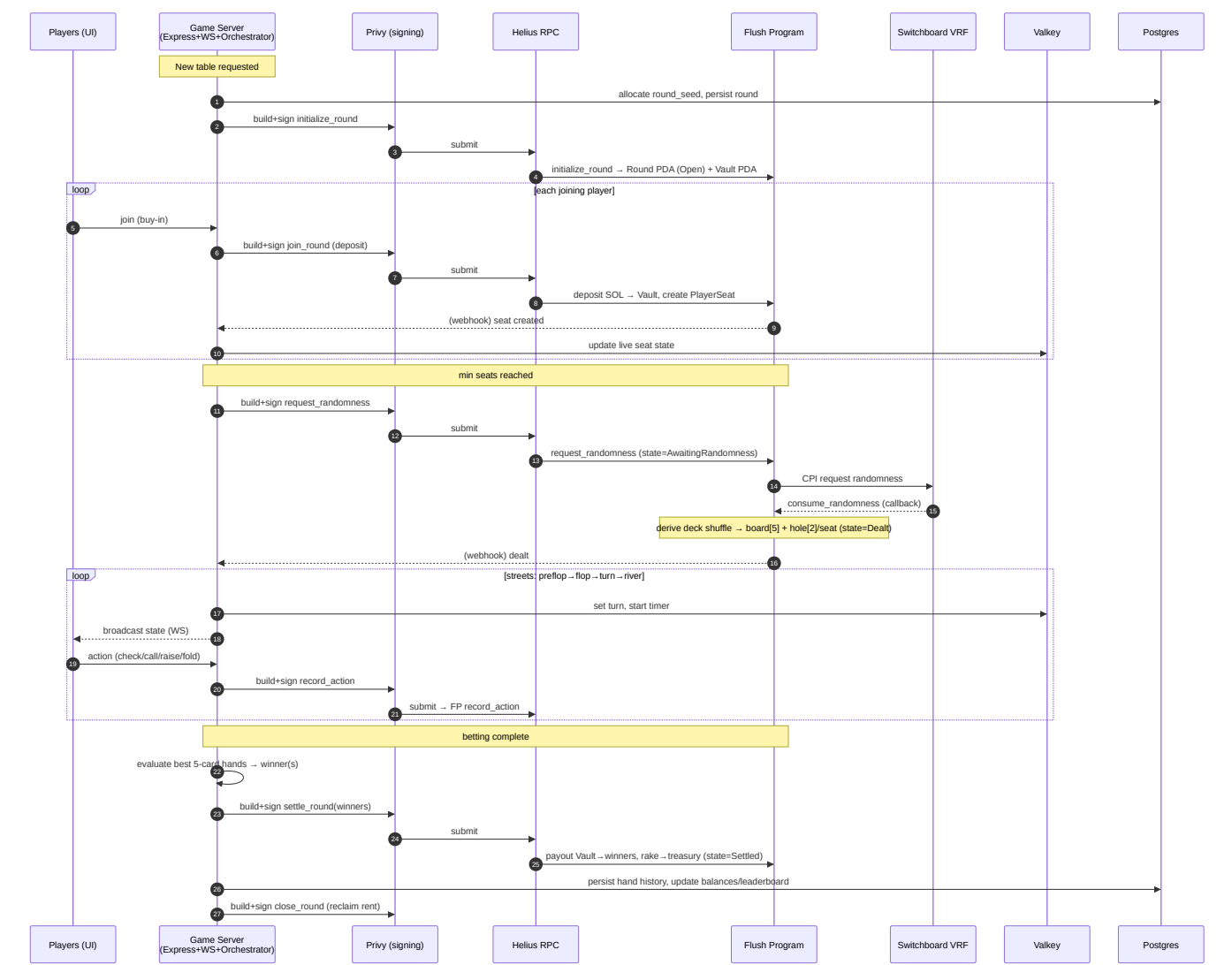


Figure 5 — End-to-end round lifecycle (sequence)

5. Instruction Flowcharts

5.1 `initialize_round`

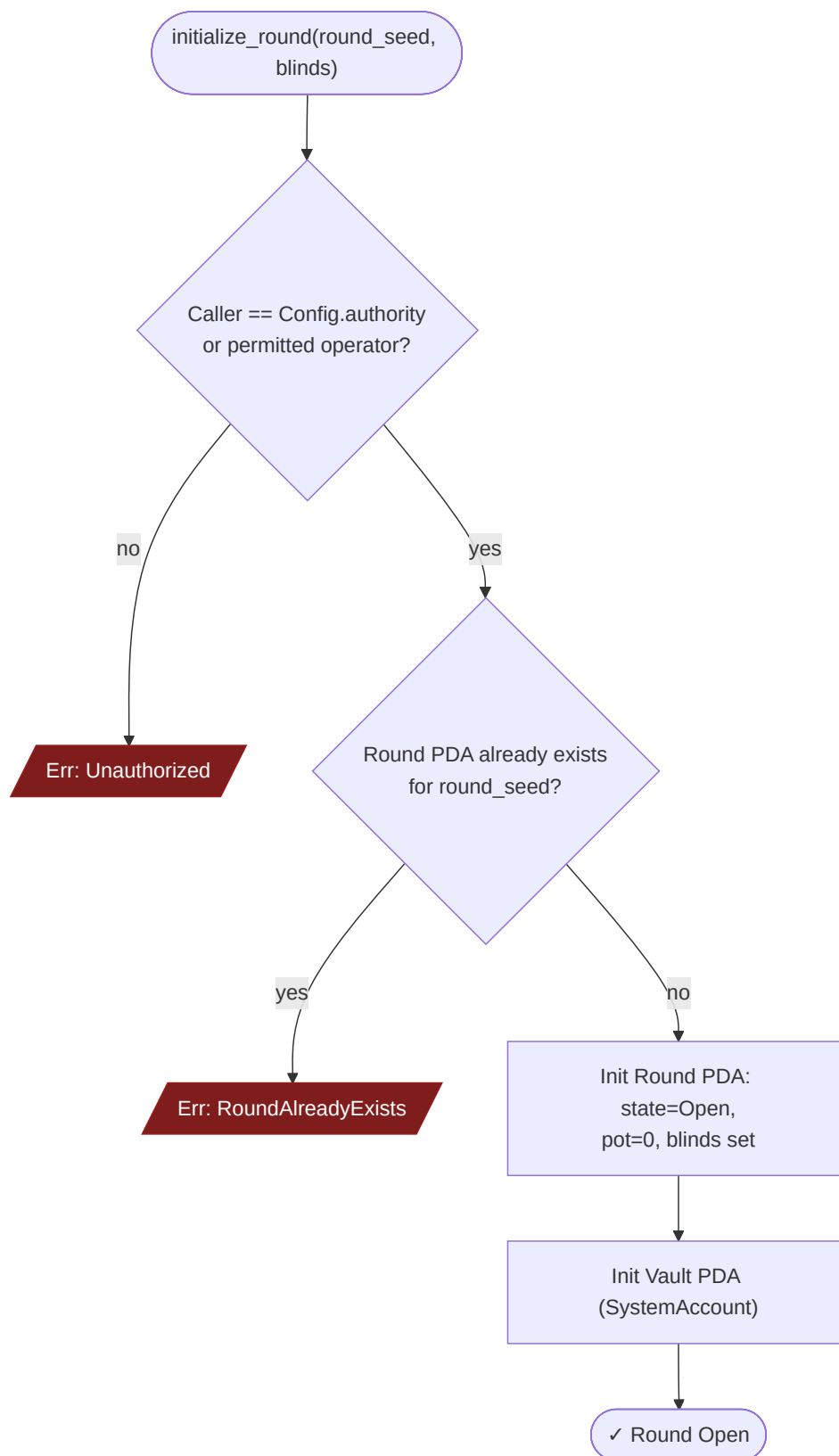


Figure 6 — `initialize_round`

5.2 join_round (deposit buy-in)

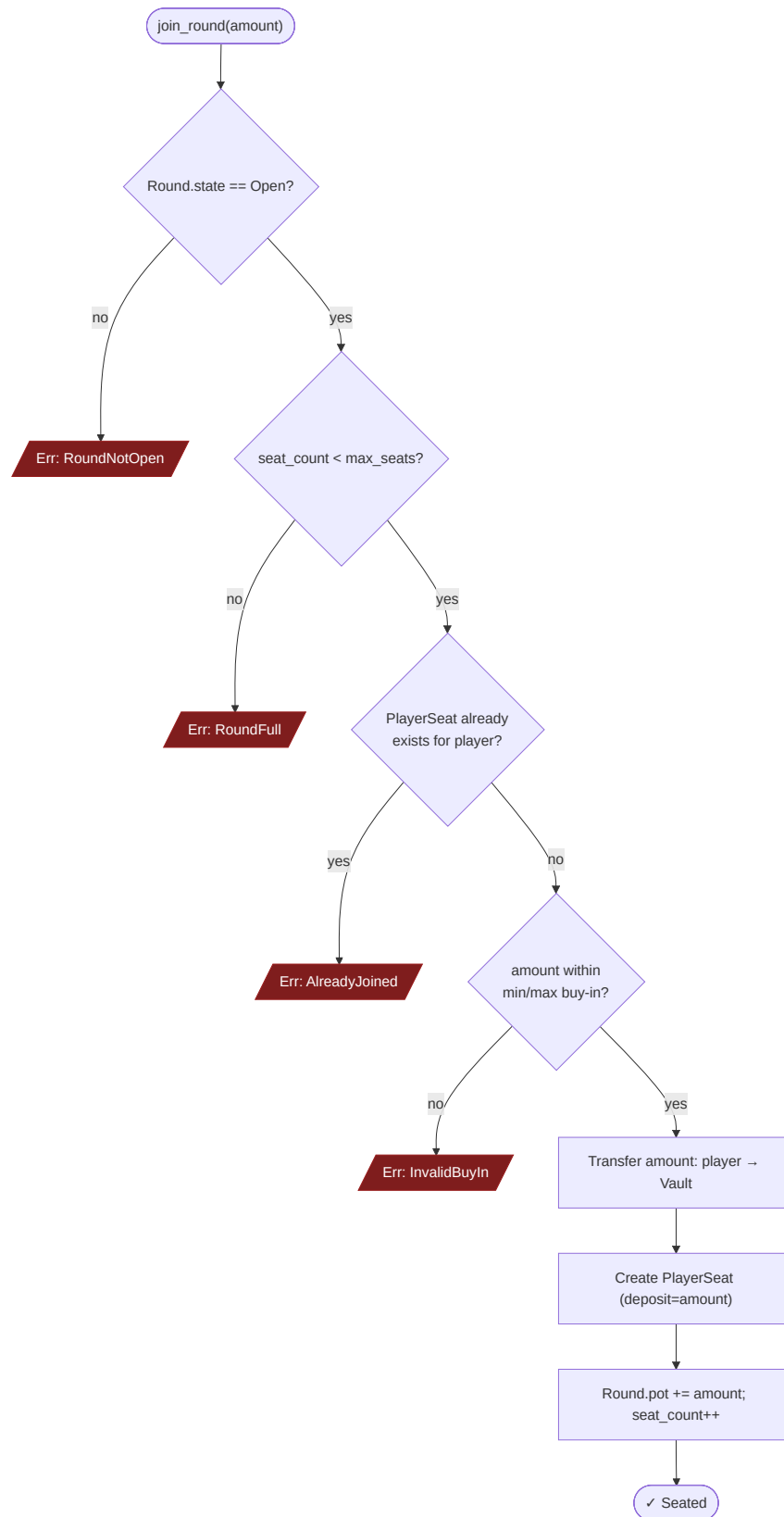


Figure 7 — join_round (deposit buy-in)

5.3 request_randomness → consume_randomness (card dealing)

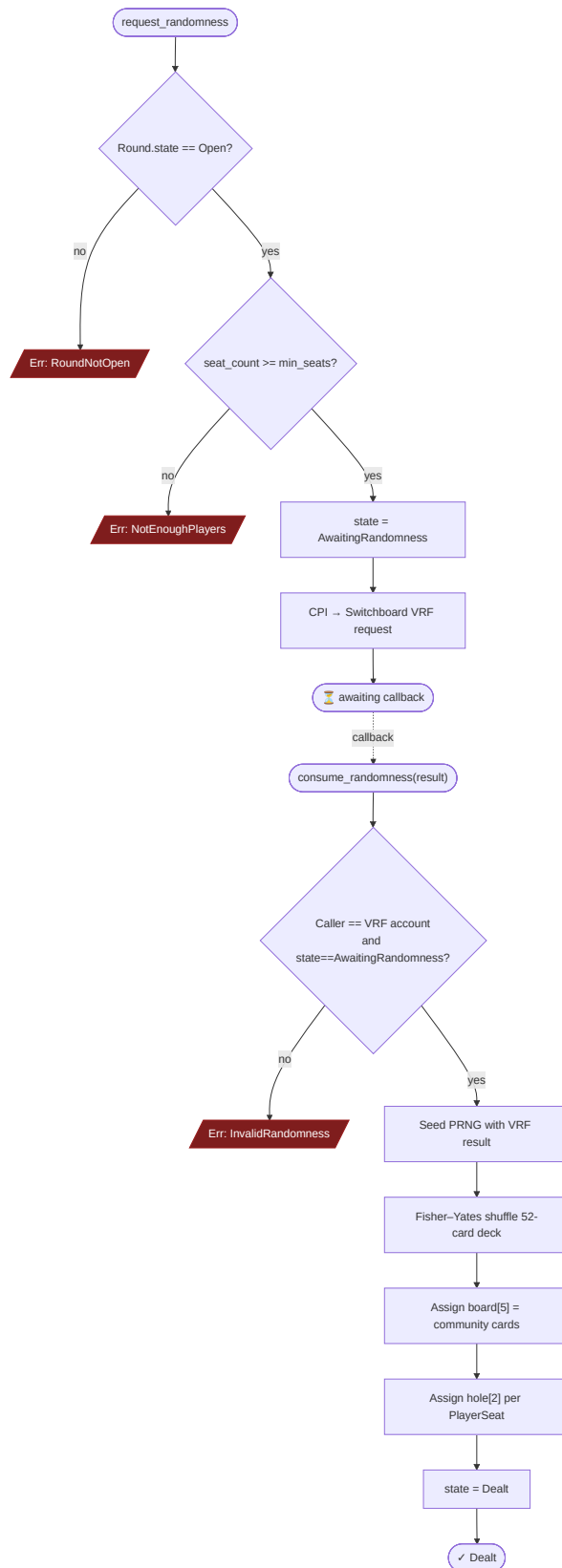


Figure 8 — request_randomness → consume_randomness (card dealing)

Card-privacy note: hole cards written on-chain are world-readable. Options to document/decide: (a) accept transparency for a fully-onchain POC, or (b) store only a **commitment/hash** on-chain and reveal at showdown, with the server holding the deal order keyed by the VRF result. The diagram shows the transparent variant; the commitment variant adds a **reveal** step at settlement.

5.4 record_action (betting)

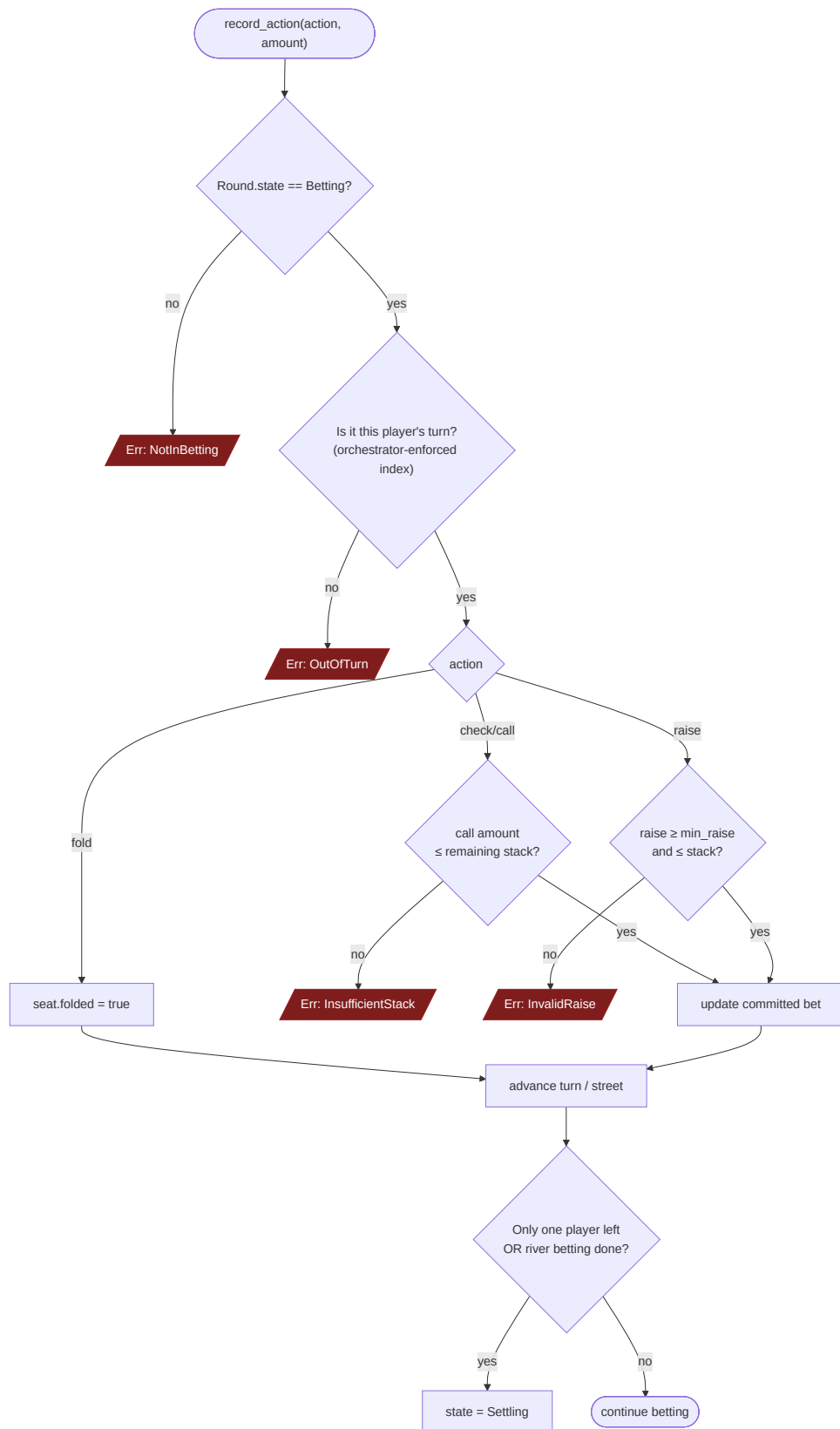


Figure 9 — `record_action (betting)`

Betting may be modeled **off-chain first** (Valkey hot state) for latency, with only the net result settled on-chain — or each action recorded on-chain for full verifiability. This is a key **trust-vs-cost decision point** to confirm.

5.5 settle_round (payout + rake)

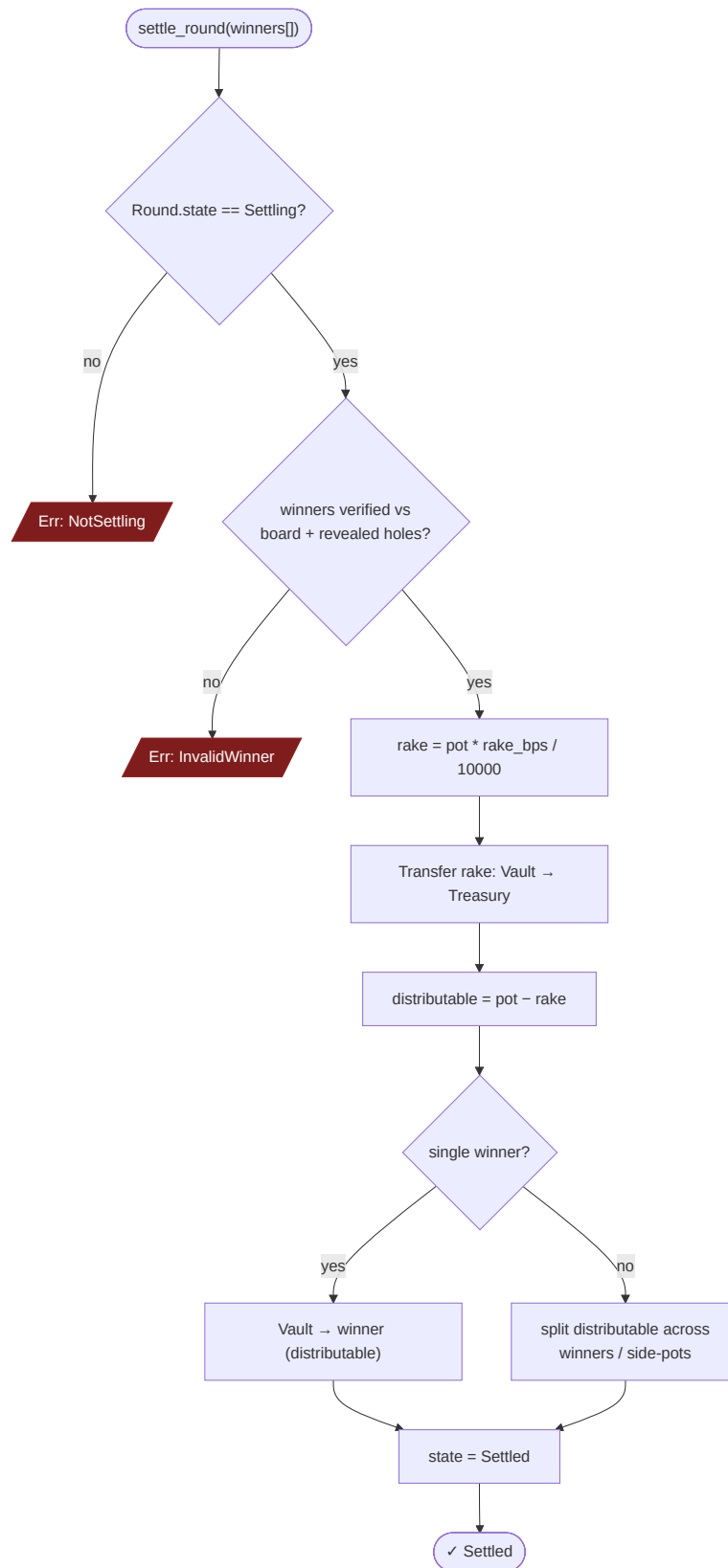


Figure 10 — settle_round (payout + rake)

5.6 close_round / close_seat

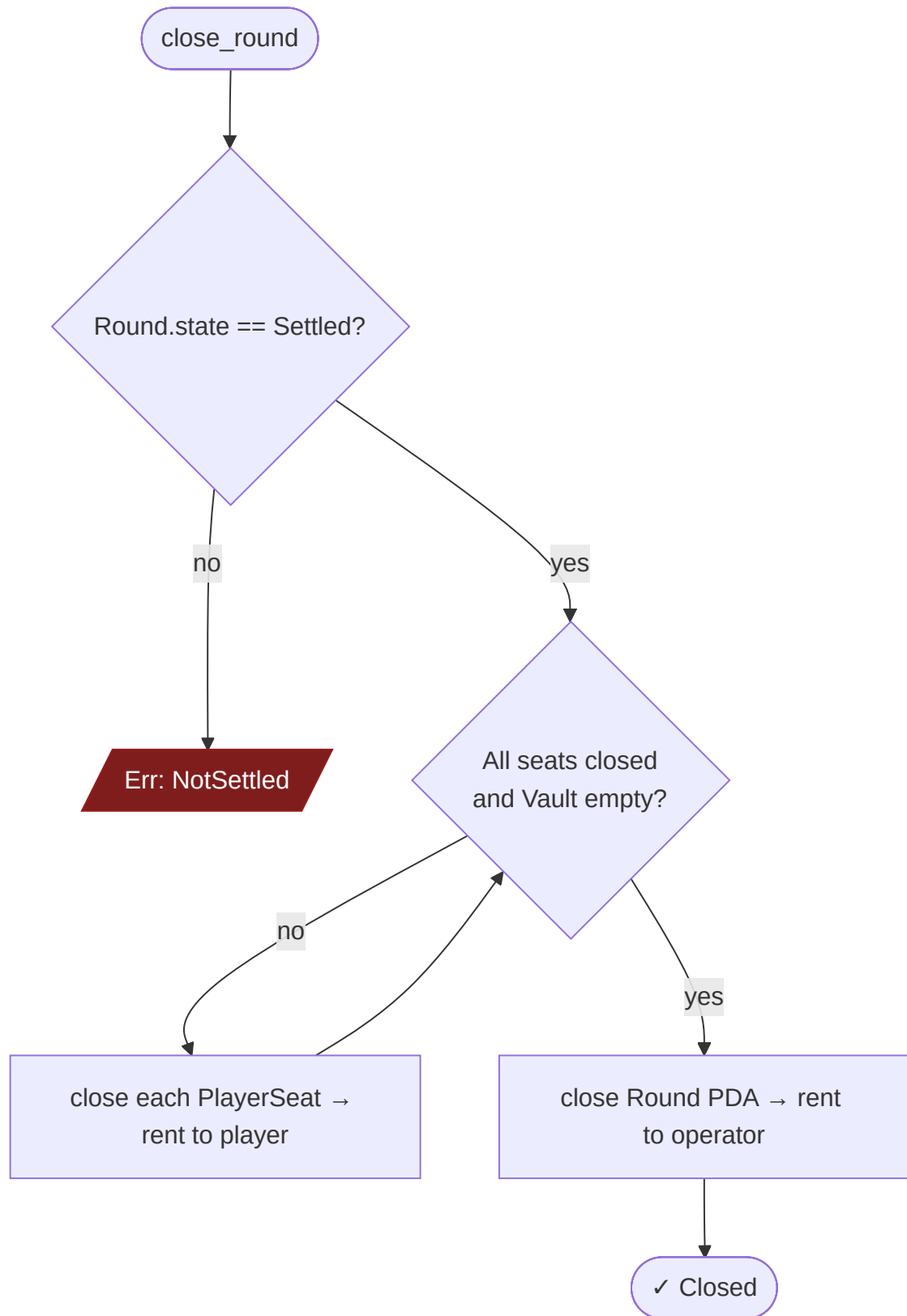


Figure 11 — close_round / close_seat

6. Randomness (VRF) Flow — Detail

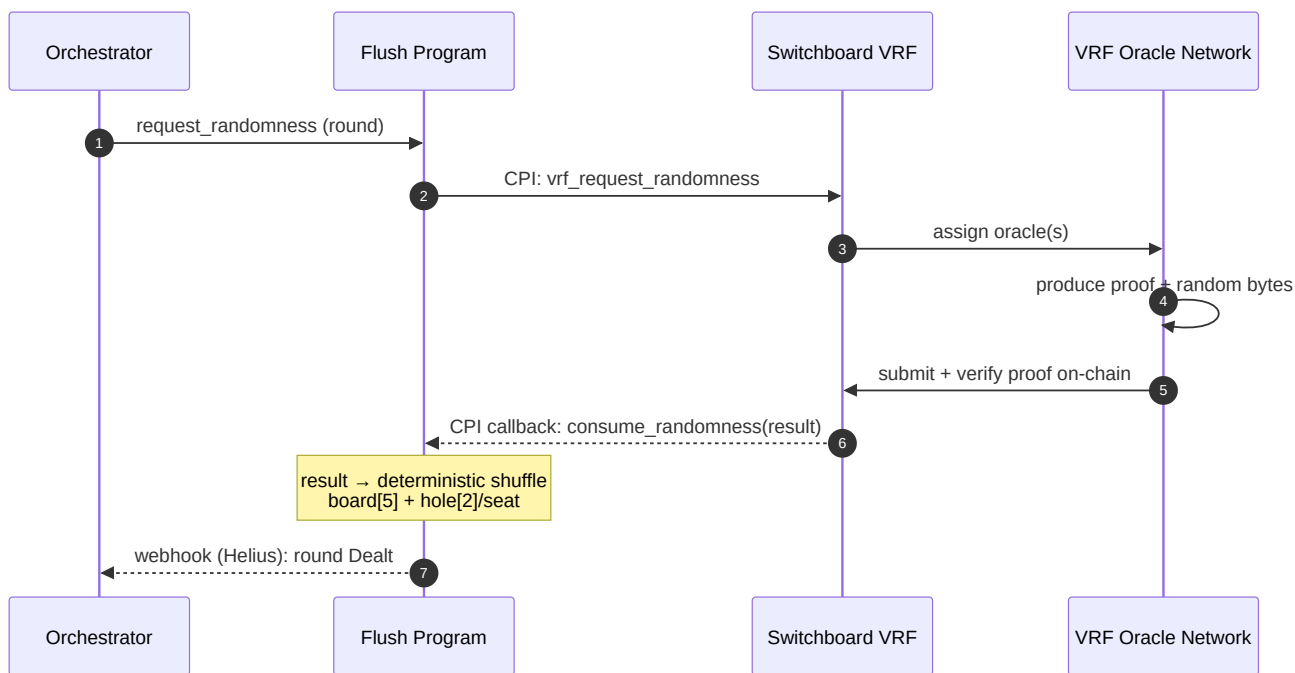


Figure 12 — VRF randomness flow (detail)

Why VRF: the deck order is provably unpredictable and verifiable — no party (server or player) can know or bias the deal before the oracle commits.

7. Data Placement — Postgres vs Valkey

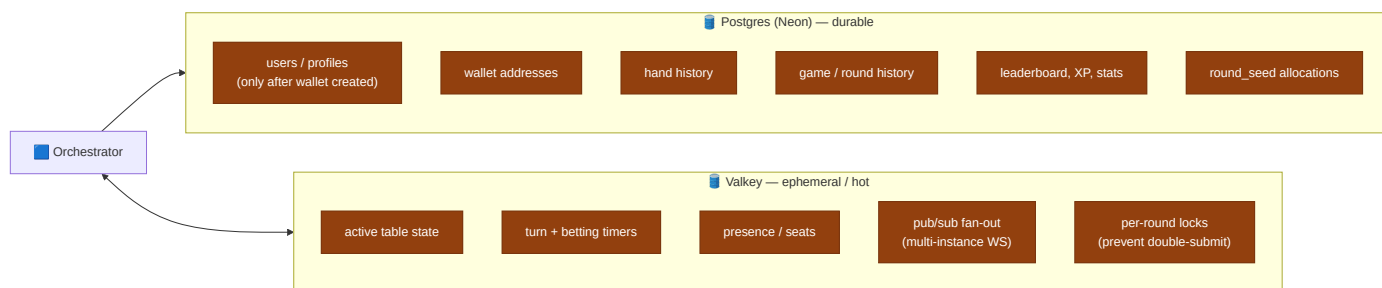


Figure 13 — Data placement: Postgres vs Valkey

- **Postgres** is the durable source of truth; profiles are persisted **only after** a Privy wallet exists (per the product rule).
- **Valkey** holds nothing that can't be rebuilt from chain + Postgres; it exists for latency and to coordinate WebSocket instances behind a load balancer.

8. Background Signing (Privy) — No User Prompts

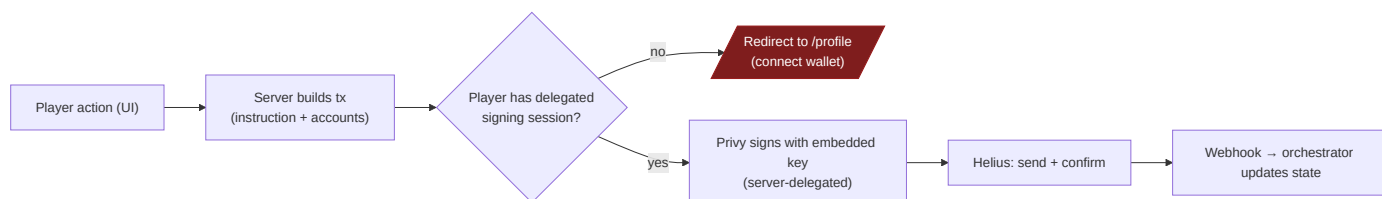


Figure 14 — Background signing (Privy), no user prompts

Funded embedded wallet + delegated session $\Rightarrow$ every poker action is just a background transaction (feels like "game points"). Only **authenticated, funded** users can play cash games; otherwise they're routed to [/profile](#) .